

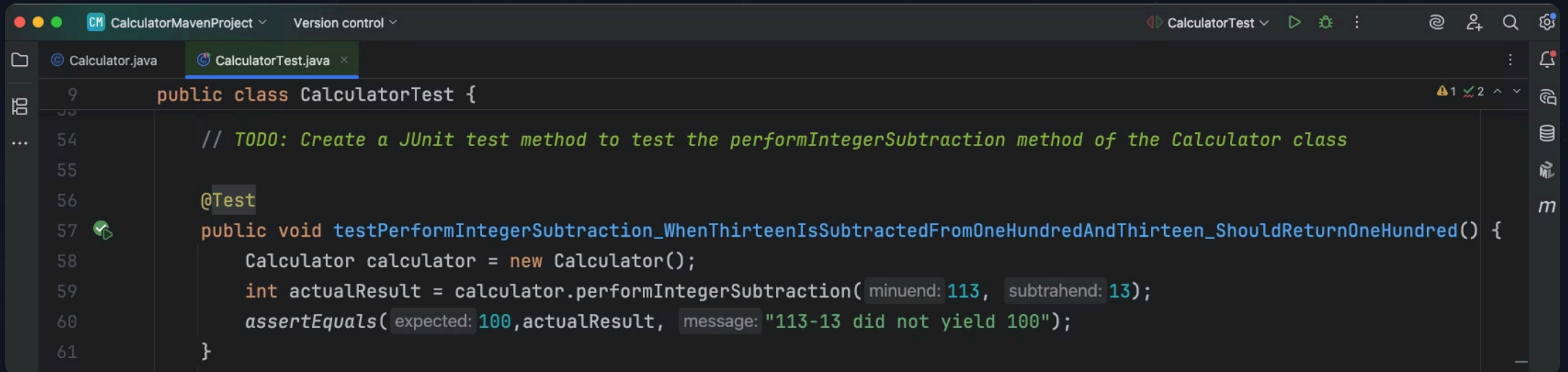
Overview of the New Subtraction Test

- A new **JUnit test** has been added to validate the `performIntegerSubtraction` **method**
- The test method is annotated with `@Test`, indicating it must be executed by **JUnit**
- Inside the test, a `Calculator` instance is created and used to call the **method under test**
- Two integers are provided: 113 (**minuend**) and 13 (**subtrahend**)
- The method returns the result of subtracting one value from the other



A simple solution

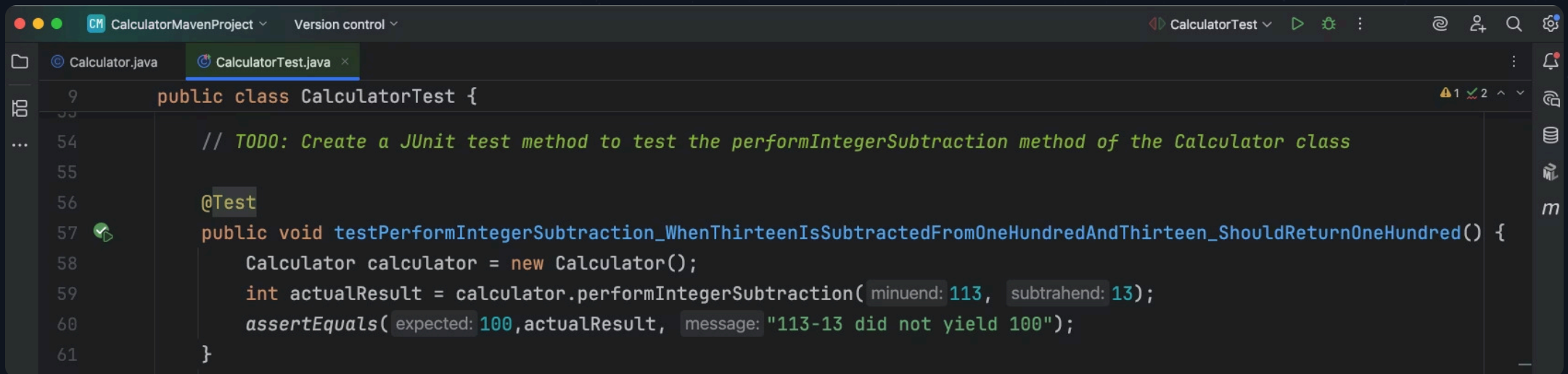
- The **actual result** of the method call is **stored** in a **local variable** (actualResult)
- The **expected outcome** for $113 - 13$ is **100**
- The **test** uses `assertEquals` to compare **expected** and **actual values**
- This ensures the **subtraction method** behaves correctly with normal input
- The **internal complexity** of the **method does not affect how the test is written**

A screenshot of an IDE window titled 'CalculatorMavenProject'. The 'CalculatorTest.java' file is open, showing a Java class with a JUnit test. The code includes a comment to create a test for the 'performIntegerSubtraction' method. The test method is annotated with '@Test' and contains logic to instantiate a 'Calculator' object, call 'performIntegerSubtraction' with minuend 113 and subtrahend 13, and then use 'assertEquals' to verify the result is 100. The IDE interface includes a sidebar with file explorer and version control, a top toolbar with run and debug buttons, and a right sidebar with search and other tools.

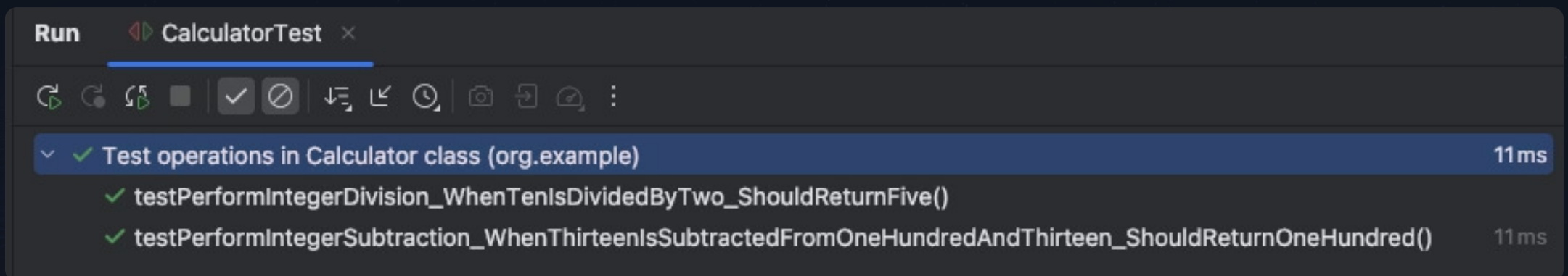
```
9 public class CalculatorTest {  
...  
54 // TODO: Create a JUnit test method to test the performIntegerSubtraction method of the Calculator class  
55  
56 @Test  
57 public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred() {  
58     Calculator calculator = new Calculator();  
59     int actualResult = calculator.performIntegerSubtraction( minuend: 113, subtrahend: 13);  
60     assertEquals( expected: 100, actualResult, message: "113-13 did not yield 100");  
61 }
```

A simple solution (contd.)

Run the tests!



```
9      public class CalculatorTest {  
10  
11      // TODO: Create a JUnit test method to test the performIntegerSubtraction method of the Calculator class  
12  
13      @Test  
14      public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred() {  
15          Calculator calculator = new Calculator();  
16          int actualResult = calculator.performIntegerSubtraction( minuend: 113, subtrahend: 13);  
17          assertEquals( expected: 100, actualResult, message: "113-13 did not yield 100");  
18      }  
19  }
```

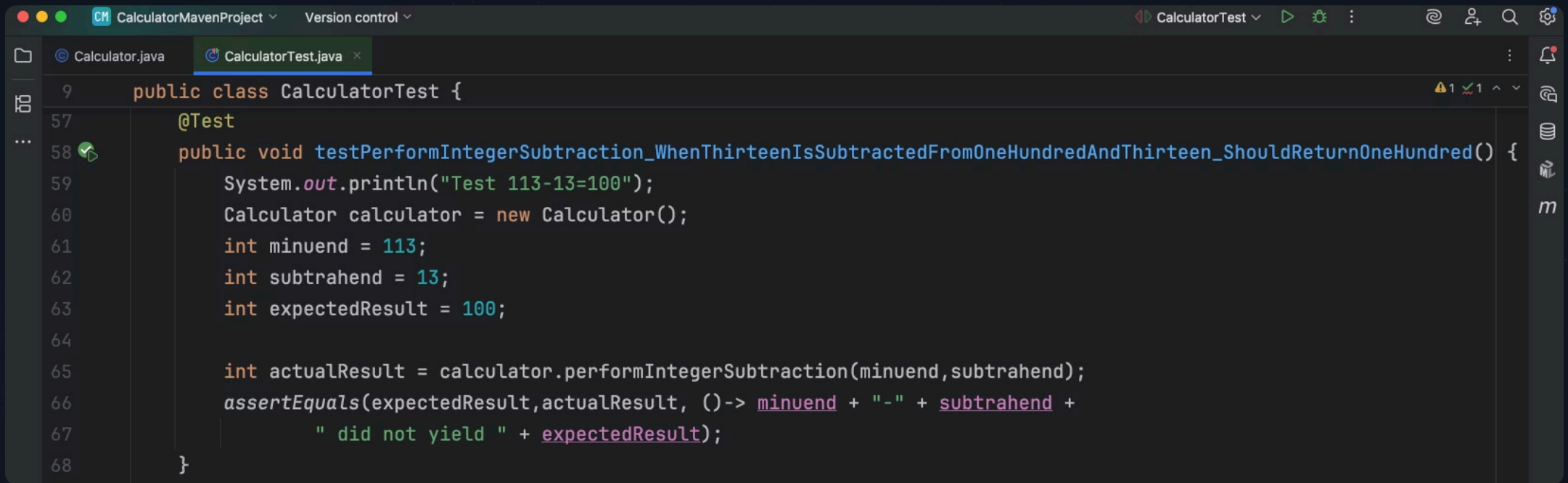


```
Run    CalculatorTest x  
✓ Test operations in Calculator class (org.example) 11 ms  
  ✓ testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive()  
  ✓ testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred() 11 ms
```

Tests passed!

An improved solution

Details of this solution, such as the use of Lambda expression will be explained in a further lesson (Lazy assertions)

A screenshot of an IDE window titled 'CalculatorMavenProject' with a 'Version control' dropdown. The editor shows two tabs: 'Calculator.java' and 'CalculatorTest.java'. The 'CalculatorTest.java' tab is active, displaying a Java test class. The code includes a package declaration, imports for 'org.junit.Test' and 'org.junit.Assert', and a single test method. The test method prints a message, creates a 'Calculator' instance, sets up test data (113 and 13), performs a subtraction, and asserts the result is 100. The IDE interface includes a sidebar with project and source views, a top toolbar with run and debug buttons, and a right sidebar with a search icon and a 'm' icon.

```
package calculator;

import org.junit.Test;
import org.junit.Assert;

public class CalculatorTest {

    @Test
    public void testPerformIntegerSubtraction_WhenThirteenIsSubtractedFromOneHundredAndThirteen_ShouldReturnOneHundred() {
        System.out.println("Test 113-13=100");
        Calculator calculator = new Calculator();
        int minuend = 113;
        int subtrahend = 13;
        int expectedResult = 100;

        int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
        assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
            " did not yield " + expectedResult);
    }
}
```

Importance of Testing with Multiple Inputs



Valid and Invalid Inputs

A method should be tested with both valid and invalid inputs



Edge Cases

Developers must consider edge cases such as very large or very small numbers



Objects and Strings

Methods that receive objects or strings should also be tested against null or unusual formats



Separate Tests

Each scenario that might break the method requires a separate test



Confidence

Multiple tests for the same method increase confidence in its correctness



Conclusion

01

Basic JUnit Structure

The subtraction test demonstrates the structure and purpose of a basic JUnit test

02

Testing Requirements

Effective testing requires checking both normal and edge-case inputs

03

Test Coverage

Independent tests for each scenario provide stronger coverage and clearer debugging information



Open Floor Discussion

Additional Test Cases

What additional test cases would you create for the subtraction method?

Edge Scenarios

Should we also test negative numbers, zero subtraction, or overflow scenarios?

Scaling Patterns

How might these patterns scale when testing more complex business logic?